

Reciprocal Rank Fusion (RRF) — Tài liệu học tập

Tài liệu giải thích thuật toán RRF được sử dụng trong hệ thống DentalAI để trộn kết quả từ Dense Retrieval (FAISS) và Sparse Retrieval (BM25).

1. RRF là gì?

Hãy tưởng tượng bạn muốn tìm phòng khám nha khoa tốt nhất. Bạn hỏi **2 người bạn** cho ý kiến:

- Bạn A** (FAISS — Vector Search): hiểu **ý nghĩa** câu hỏi, tìm tài liệu có nội dung tương đồng về mặt ngữ nghĩa.
- Bạn B** (BM25 — Keyword Search): tìm tài liệu có chứa đúng **từ khóa** bạn nói.

Mỗi người đưa ra một danh sách xếp hạng riêng. Vấn đề là: **làm sao gộp 2 danh sách thành 1 danh sách duy nhất một cách công bằng?**

RRF (Reciprocal Rank Fusion) chính là phương pháp để làm việc đó — nó lấy **thứ hạng** từ mỗi danh sách, chuyển thành điểm số chung, rồi cộng lại.

Tại sao không cộng thẳng điểm số (raw score)?

Vì điểm số của FAISS và BM25 **hoàn toàn khác bản chất**, không thể so sánh trực tiếp:

	FAISS (Cosine Similarity)	BM25
Thang điểm	0.0 → 1.0	0 → hàng chục, không giới hạn
Phân bố	Dồn quanh 0.7–0.9	Phân tán rộng, phụ thuộc độ dài tài liệu
Ý nghĩa	Góc giữa 2 vector	Tần suất từ khóa (TF-IDF variant)

Cộng 0.85 (FAISS) + 12.7 (BM25) = 13.55 → **BM25 áp đảo hoàn toàn**, FAISS gần như vô nghĩa. Đây là lý do ta cần RRF: **chỉ dùng thứ hạng, bỏ qua điểm số thô**, đưa mọi thứ về cùng một thang đo.

2. Phân tích công thức RRF

Công thức gốc (bài báo Cormack 2009)

$$\text{RRF_score}(\text{doc}) = \sum \frac{1}{(k + \text{rank_i})}$$

Trong hệ thống DentalAI, công thức được mở rộng thêm **trọng số động** (weighted RRF):

$$\begin{aligned} \text{RRF_score}(\text{doc}) = & w_{\text{vector}} / (k + \text{rank_faiss} + 1) \\ & + w_{\text{bm25}} / (k + \text{rank_bm25} + 1) \end{aligned}$$

Tương ứng với code thực tế trong [src/retriever/search.py](#):

```
scores[idx] = (
    w_vector / (self._RRF_K + v_r + 1)
    + w_bm25 / (self._RRF_K + b_r + 1)
)
```

Ý nghĩa từng thành phần

rank — thứ hạng của tài liệu trong danh sách (bắt đầu từ 0). Rank = 0 nghĩa là đứng đầu.

$1 / (k + \text{rank} + 1)$ — nghịch đảo thứ hạng. Tài liệu đứng **càng cao** → rank càng nhỏ → điểm **càng lớn**. Tài liệu đứng **thấp hoặc không xuất hiện** → rank rất lớn → điểm **gần 0**.

k = 60 — hằng số làm mượt (smoothing constant). Đây là phần quan trọng nhất, giải thích ở mục tiếp theo.

Tại sao có +1 trong mẫu số?

Công thức gốc Cormack 2009 dùng rank bắt đầu từ **1** (1-indexed): $1 / (k + \text{rank})$ với $\text{rank} \in \{1, 2, 3, \dots\}$.

Trong code DentalAI, rank bắt đầu từ **0** (0-indexed) vì Python enumerate/argsort trả về chỉ số từ 0. Nếu giữ nguyên $1 / (k + \text{rank})$ thì tài liệu rank 0 sẽ cho mẫu số = k thay vì k+1, **phá vỡ tính tương đương** với công thức gốc.

Thêm **+1** để chuyển 0-indexed → 1-indexed ngay trong công thức:

Công thức gốc (1-indexed):	$1 / (k + \text{rank}_1)$	với $\text{rank}_1 = 1, 2, 3, \dots$
Code (0-indexed):	$1 / (k + \text{rank}_0 + 1)$	với $\text{rank}_0 = 0, 1, 2, \dots$

Hai công thức **hoàn toàn tương đương** vì $\text{rank}_1 = \text{rank}_0 + 1$. Đây chỉ là sự khác biệt quy ước chỉ số, không phải thay đổi thuật toán.

Tương ứng trong code ([src/retriever/search.py](#)):

```
scores[idx] = (
    w_vector / (self._RRF_K + v_r + 1)  # +1 vì v_r bắt đầu từ 0
    + w_bm25 / (self._RRF_K + b_r + 1)  # +1 vì b_r bắt đầu từ 0
)
```

Miss Rank — Khi tài liệu không xuất hiện trong 1 danh sách

Khi một tài liệu chỉ được tìm thấy bởi FAISS mà **không xuất hiện** trong kết quả BM25 (hoặc ngược lại), nó không có thứ hạng ở danh sách còn lại. Hệ thống xử lý bằng cách gán một **Miss Rank = 1000**

(`_RRF_MISS_RANK = 1000`):

```
v_r = v_rank_map.get(idx, self._RRF_MISS_RANK) # không có trong FAISS → 1000
b_r = b_rank_map.get(idx, self._RRF_MISS_RANK) # không có trong BM25 → 1000
```

Tại sao 1000 chứ không phải vô cực (∞)?

- Nếu dùng $\infty \rightarrow 1/(k + \infty) = 0$, danh sách bị miss đóng góp **đúng 0 điểm**. Quá khắc nghiệt — một tài liệu rank 1 ở FAISS nhưng miss BM25 sẽ bị đối xử như chỉ có 1 nguồn tin cậy.
- Nếu dùng $1000 \rightarrow 1/(60 + 1000 + 1) = 0.000943$, đóng góp **gần 0 nhưng không hoàn toàn 0**. Tài liệu vẫn giữ một lượng điểm nhỏ từ danh sách miss, tránh bị loại oan nếu rank ở danh sách kia rất cao.
- Con số 1000 được chọn đủ lớn để tạo "hình phạt nặng" (điểm thành phần giảm ~17 lần so với rank 0) nhưng không triệt tiêu hoàn toàn.

Ví dụ so sánh:

Trường hợp	rank FAISS	rank BM25	Điểm từ FAISS	Điểm từ BM25	Tổng
Có cả 2 (rank 0 + rank 0)	0	0	$1/61 = 0.01639$	$1/61 = 0.01639$	0.03279
FAISS rank 0, BM25 miss	0	1000	$1/61 = 0.01639$	$1/1061 = 0.00094$	0.01733
BM25 miss hoàn toàn (∞)	0	∞	$1/61 = 0.01639$	$1/\infty = 0.00000$	0.01639

Tài liệu xuất hiện ở **cả 2 danh sách** luôn có lợi thế rõ rệt — đây chính là triết lý cốt lõi của RRF: **đồng thuận giữa nhiều nguồn đáng tin hơn một nguồn đơn lẻ**.

Vai trò của hằng số k

Không có k, công thức trở thành $1/\text{rank}$ — tài liệu rank 1 được **1.0 điểm**, rank 2 chỉ được **0.5 điểm** (giảm 50%). Chênh lệch quá lớn, top 1 áp đảo tất cả.

Khi thêm k = 60:

- Rank 0: $1/(60+0+1) = 0.01639$
- Rank 1: $1/(60+1+1) = 0.01613$
- Rank 2: $1/(60+2+1) = 0.01587$

Chênh lệch giữa các vị trí chỉ còn **rất nhỏ** → tài liệu rank 5 vẫn có cơ hội cạnh tranh với rank 1 nếu nó xuất hiện ở **cả 2 danh sách**.

k thay đổi thì sao?

k	Hiệu ứng	Khi nào dùng
k = 10 (nhỏ)	Top 1-2 được ưu tiên mạnh, tài liệu rank thấp gần như bị loại	Khi bạn rất tin vào thứ hạng ban đầu

k	Hiệu ứng	Khi nào dùng
k = 60 (mặc định)	Cân bằng — top cao vẫn được ưu tiên nhưng không áp đảo	Phổ biến nhất , phù hợp đa số trường hợp
k = 100 (lớn)	Mọi vị trí gần như bằng nhau, RRF chủ yếu đếm "xuất hiện ở bao nhiêu danh sách"	Khi bạn không tin lắm vào ranking riêng lẻ

3. Ví dụ minh họa — Tính tay

Giả sử query: **"chi phí niềng răng"** → k = 60, w_vector = 0.3, w_bm25 = 0.7 (câu hỏi keyword-heavy).

Kết quả từ 2 kênh

Tài liệu	Rank FAISS	Rank BM25
Doc A — "Niềng răng - Tổng quan"	0 (top 1)	2
Doc B — "Niềng răng - Chi phí"	2	0 (top 1)
Doc C — "Invisalign - Bảng giá"	1	không có (miss → rank = 1000)

Tính RRF score từng doc

Doc A:

$$\begin{aligned} &= 0.3 / (60 + 0 + 1) + 0.7 / (60 + 2 + 1) \\ &= 0.3 / 61 + 0.7 / 63 \\ &= 0.004918 + 0.011111 \\ &= 0.016029 \end{aligned}$$

Doc B:

$$\begin{aligned} &= 0.3 / (60 + 2 + 1) + 0.7 / (60 + 0 + 1) \\ &= 0.3 / 63 + 0.7 / 61 \\ &= 0.004762 + 0.011475 \\ &= 0.016237 \leftarrow \text{CAO NHAT} \end{aligned}$$

Doc C: (không xuất hiện trong BM25 → rank = 1000)

$$\begin{aligned} &= 0.3 / (60 + 1 + 1) + 0.7 / (60 + 1000 + 1) \\ &= 0.3 / 62 + 0.7 / 1061 \\ &= 0.004839 + 0.000660 \\ &= 0.005499 \leftarrow \text{THAP NHAT} \end{aligned}$$

Kết quả xếp hạng cuối cùng

Hạng	Tài liệu	RRF Score	Phân tích
1	Doc B — Chi phí	0.016237	BM25 rank 1 + trọng số BM25 cao (0.7) → chiến thắng
2	Doc A — Tổng quan	0.016029	Xuất hiện ở cả 2 danh sách, nhưng BM25 rank thấp hơn
3	Doc C — Invisalign	0.005499	Chỉ có FAISS tìm thấy, BM25 miss → bị phạt nặng

Nhận xét: Doc C dù FAISS rank 1 (rất cao) nhưng vì BM25 không tìm thấy nó (miss rank = 1000) và trọng số BM25 chiếm 0.7, nên nó rớt xuống cuối. Đây chính là cơ chế **chống ám thị chi tiết** — bài quảng cáo thương hiệu cụ thể bị đẩy xuống.

Tài liệu tham khảo

- Cormack, G. V., Clarke, C. L. A., & Buettcher, S. (2009). *Reciprocal Rank Fusion outperforms Condorcet and individual Rank Learning Methods*. SIGIR 2009.